
Übungsblatt 3

Abgabe: 11.05.2026, 10:00 Uhr (im Digicampus via VIPS: .java-Dateien für Code, .uxf für UML, .pdf für alles andere)

- Dieses Übungsblatt muss im Team abgegeben werden (Einzelabgaben sind nicht erlaubt!).
- Die **Zeitangaben** geben zur Orientierung an, wie viel Zeit für eine Aufgabe später in der Klausur vorgesehen wäre; gehen Sie davon aus, dass Sie zum jetzigen Zeitpunkt wesentlich länger brauchen und die angegebene Zeit erst nach ausreichender Übung erreichen.

* leichte Aufgabe / ** mittelschwere Aufgabe / *** schwere Aufgabe

Aufgabe 9 ** (*Eigene Klasse implementieren, 29 Minuten*)

In dieser Aufgabe sollen Sie eine Klasse **Student** implementieren, mit der Studierende einer Universität verwaltet werden. Jeder Studierende hat einen **Namen** (nichtleere Zeichenkette), eine **Matrikelnummer** (positive siebenstellige ganze Zahl) sowie ein aktuelles **Fachsemester** (ganze Zahl zwischen 1 und 20, jeweils eingeschlossen). Außerdem existiert eine **Regelstudienzeit** von 6 Semestern, die für alle Studierenden gleich ist und nicht geändert werden kann. Es sollen Methoden zur Verfügung gestellt werden, um diese Informationen auszulesen und zu bearbeiten. Zudem soll eine Methode **isInStandardStudyPeriod()** bereitgestellt werden, die entscheidet, ob sich ein Studierender noch innerhalb der Regelstudienzeit befindet.

a) (*, *Klassenkarte entwerfen, 8 Minuten*)

Entwerfen Sie eine Klassenkarte für die Klasse **Student**.

b) (**, *Klasse implementieren, 15 Minuten*)

Implementieren Sie die Klasse **Student**. Vergessen Sie nicht den Konstruktor sowie passende Datentypen.

- Aufgrund des Einkapselungsprinzips sollen alle Attribute privat sein und ausschließlich über **get**- und **set**-Methoden ausgelesen bzw. verändert werden können.
- Für jedes der drei Attribute sollen die **set**-Methoden vor dem Setzen mithilfe einer privaten **Check**-Methode überprüfen, ob der übergebene Wert gültig ist:
 - Der Name muss eine nichtleere Zeichenkette sein.
 - Die Matrikelnummer muss eine positive siebenstellige ganze Zahl sein.
 - Das Fachsemester muss zwischen 1 und 20 liegen (jeweils eingeschlossen).

Ist der übergebene Wert ungültig, soll das Attribut **nicht** verändert werden.

- Verwenden Sie die zuvor implementierten **set**-Methoden im Konstruktor.
- Implementieren Sie auch die in a) entworfene Methode **isInStandardStudyPeriod()**, die zurückgibt, ob das aktuelle Fachsemester kleiner oder gleich der Regelstudienzeit ist.

- Die Regelstudienzeit soll **nicht** geändert, aber trotzdem ausgelesen werden können.

c) (*, Programmklasse implementieren, 6 Minuten)

Implementieren Sie eine Programmklasse `University`, mit der Sie Ihre `Student`-Klasse testen können:

- Erzeugen Sie einen Studierenden `s1` mit Namen "Mikail Melnyk", Matrikelnummer 1234567 und Fachsemester 3.
- Erzeugen Sie einen Studierenden `s2` mit Namen "Lara Yilmaz", Matrikelnummer 7654321 und Fachsemester 8.
- Versuchen Sie, das Fachsemester von `s1` auf `-1` zu setzen und geben Sie anschließend das Fachsemester von `s1` auf der Kommandozeile aus.
- Erhöhen Sie das Fachsemester von `s1` um 1.
- Geben Sie für beide Studierenden aus, ob sie sich noch innerhalb der Regelstudienzeit befinden.

Hinweis: Nutzen Sie ausschließlich Methoden der Klasse `Student`, um auf Attribute zuzugreifen.

Aufgabe 10 + 11 (Eigene Datenklassen modellieren und implementieren, 40 Minuten)

In dieser Aufgabe sollen Sie drei Klassen zum Verwalten von Fahrzeugen für eine Fahrzeugvermietung nach folgender Systembeschreibung modellieren und implementieren:

*Es gibt keine allgemeinen Fahrzeuge, sondern nur Autos und LKWs. Trotzdem soll die Möglichkeit offen gehalten werden, das Programm später um zusätzliche Fahrzeugtypen zu erweitern. Implementieren Sie deshalb eine **abstrakte** Klasse `Vehicle` und zwei Unterklassen `Car` und `Truck` dieser abstrakten Klasse. Jedes Fahrzeug hat ein Kennzeichen und stellt Möglichkeiten zur Verfügung, dieses zu lesen und zu ändern. Außerdem erfordert die Klasse `Vehicle`, dass jede Unterklasse eine Möglichkeit zur Verfügung stellt, den reellwertigen Mietpreis des Fahrzeuges zu bestimmen. Weil das für allgemeine Fahrzeuge nicht sinnvoll ist, soll diese Methode abstrakt sein. Zusätzlich wird gespeichert, wie viele Fahrzeuge insgesamt aktuell verwaltet werden. Es soll auch eine Methode angeboten werden, um die aktuelle Anzahl an verwalteten Fahrzeugen zu erhalten. Die Darstellung eines Fahrzeugs als Zeichenkette sieht wie folgt aus:*

"License plate: A-UA-1234"

Ein Auto hat eine ganzzahlige Anzahl an Sitzplätzen und stellt Möglichkeiten zur Verfügung, diese anzupassen und abzufragen. Der Mietpreis eines Autos ist das 2.5-fache seiner Anzahl an Sitzplätzen. Ein LKW hat ein reellwertiges Maximalgewicht und stellt Möglichkeiten zur Verfügung, dieses anzupassen und abzufragen. Der Mietpreis eines LKWs ist das 10-fache seines Maximalgewichts. Die Zeichenkettendarstellung der beiden Unterklassen sieht wie folgt aus:

"License plate: A-UA-1234 Number of seats: 4 Rental price: 10.0"

"License plate: A-UA-1235 Maximum weight: 40.0 Rental price: 400.0"

a) (*, Klassenkarte entwerfen, 12 Minuten)

Entwerfen Sie passende Klassenkarten für Fahrzeuge, Autos und LKWs. Achten Sie darauf, das Verhältnis zwischen Fahrzeugen und Autos bzw. LKWs korrekt darzustellen.

Hinweis: Das Überschreiben von Methoden in Unterklassen modellieren wir, indem wir die zu überschreibende Methode auch in der Klassenkarte der Unterklasse aufführen.

b) (***, Klassen implementieren, 23 Minuten*)

Implementieren Sie die Klassen **Vehicle**, **Car** und **Truck**. Versehen Sie auch die Klasse **Vehicle** mit einem Konstruktor, den Sie in den Konstruktoren von **Car** und **Truck** verwenden. Überschreiben Sie die **toString()**-Methode der Klasse **Vehicle** und verwenden Sie diese in der Implementierung der **toString()**-Methoden von **Car** und **Truck**.

c) (***, Programmklasse implementieren, 5 Minuten*)

Implementieren Sie eine Programmklasse, die

- ein **Vehicle**-Array **vehicles** der Größe 2 anlegt,
- ein Auto mit dem Kennzeichen **"A-UA-1234"** und 4 Sitzen anlegt und an der ersten Stelle des Arrays speichert,
- einen LKW mit dem Kennzeichen **"A-UA-1235"** und Maximalgewicht 40.0 anlegt und an der zweiten Stelle des Arrays speichert und
- die Zeichenkettendarstellung aller Fahrzeuge im Array mit einer Schleife ausgibt.

Aufgabe 12 (*Eigene Klasse implementieren & die Klasse **ArrayList**, 25 Minuten*)

In dieser Aufgabe lernen Sie die Klasse **ArrayList** kennen. Mit einem **ArrayList**-Objekt kann man Listen von Objekten verwalten. Im Unterschied zu Arrays wie z.B. **new String[5]**, die immer eine feste Länge haben, passen sich **ArrayList**-Objekte ihrer Größe dynamisch während der Programmlaufzeit an. **ArrayLists** bieten viele weitere nützliche Funktionen an, die wir in Kapitel 5 näher kennen lernen werden. Für diese Aufgabe reicht der Hinweis, dass wir **ArrayList**-Variablen *parametrisieren müssen*: Wenn wir ein **ArrayList**-Objekt für **String**-Objekte erzeugen wollen, geht das mit der Anweisung **ArrayList<String> myStringList = new ArrayList<>();** Verwenden Sie die in Digicampus zur Verfügung gestellte Klasse **FreighterMain**, um Ihre Implementierung zu testen.

a) (**, Klassenkarten entwerfen, 6 Minuten*)

Entwerfen Sie zwei Klassenkarten: **FreightContainer** für Frachtcontainer und **Freighter** für Frachtschiffe. Alle Frachtcontainer haben dasselbe Leergewicht von 3900.0. Außerdem haben Frachtcontainer ein reellwertiges Ladungsgewicht. Ein Frachtcontainer bietet mit **setCargoWeight()** eine Methode an, das Ladungsgewicht zu ändern und mit **getTotalWeight()** eine Methode, um sein Gesamtgewicht, bestehend aus Leergewicht + Ladungsgewicht, abzufragen.

Ein Frachtschiff hat ein maximales Gewicht, mit dem es beladen werden kann. Das maximale Gewicht des Frachtschiffs kann nicht mehr verändert werden. Außerdem kann ein Frachtschiff keinen, einen oder mehrere Container geladen haben. Ein Frachtschiff bietet mit **addContainer()** bzw. **removeContainer()** Methoden an, einen Container zu entfernen bzw. hinzuzufügen. Außerdem bietet ein Frachtschiff mit **calculateContainerWeight()** eine Methode an, das Gesamtgewicht aller geladenen Container zu bestimmen.

b) (**, **ArrayList** kennenlernen, 5 Minuten*)

Recherchieren Sie in der API vier Methoden der Klasse **ArrayList** und notieren Sie hier ihre Methodenköpfe:

- Eine Methode, um die Anzahl der enthaltenen Objekte zu bestimmen
- Eine Methode, um ein Objekt an einer bestimmten Position zu erhalten
- Eine Methode, um ein Objekt hinzuzufügen
- Eine Methode, um ein Objekt an einer bestimmten Position zu entfernen

Hinweis: In der API werden Sie bei der Klasse **ArrayList** bei ganz vielen Methoden den Typ **E** sehen. Das ist der Typ, mit dem das **ArrayList**-Objekt parametrisiert wurde. In unserem Fall erwartet es dann ein Frachtcontainer-Objekt bzw. gibt eines zurück usw.

c) (**, Klassen implementieren, 14 Minuten)

Implementieren Sie die Klassen für Frachtcontainer bzw. Frachtschiffe:

- Sorgen Sie dafür, dass Objekte der Klasse **FreightContainer** sowohl mit als auch ohne Angabe eines initialen Ladungsgewichts (Default: 0) erzeugt werden können.
- Verwenden Sie für die Verwaltung der auf einem Frachtschiff geladenen Container eine **ArrayList** für **FreightContainer**-Objekte.

Stellen Sie sich die **ArrayList** wie einen Mitarbeiter vor, der für die Frachtcontainer zuständig ist und verwenden Sie die zuvor recherchierten Methoden:

- Um zu bestimmen, wie viel alle Container auf dem Schiff zusammen wiegen, fragen wir den zuständigen Mitarbeiter (also die **ArrayList**), wie viele Container wir haben, gehen alle enthaltenen Container der Reihe nach durch und addieren die Gewichte der einzelnen Container.
- Wenn das Schiff einen Container entfernen soll, übergibt es den Container an den zuständigen Mitarbeiter, der diese Aufgabe übernimmt.
- Beim Hinzufügen eines Containers soll überprüft werden, ob der neue Container zu schwer ist: Nur, wenn das zukünftige Gesamtgewicht nicht oberhalb des Maximalgewichts des Schiffs liegen würde, darf der Container hinzugefügt werden. Verwenden Sie einen geeigneten Rückgabetypen, um klar zu machen, ob der Container hinzugefügt wurde oder nicht.